

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра вычислительной техники

ОТЧЁТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
ПО ДИСЦИПЛИНЕ «ПРОГРАММИРОВАНИЕ»

Факультет	ЗО	Преподаватель	Дубков Илья
Группа	ДТ-460а		Сергеевич
Студент	Пантюхин Артём Евгеньевич		
Вариант	9		

Новосибирск, 2026 г.

Цель работы

Изучить работу потоков ввода-вывода и реализацию перегрузки потоков ввода-вывода на стандартные устройства и в файл для разработанных классов.

Задание

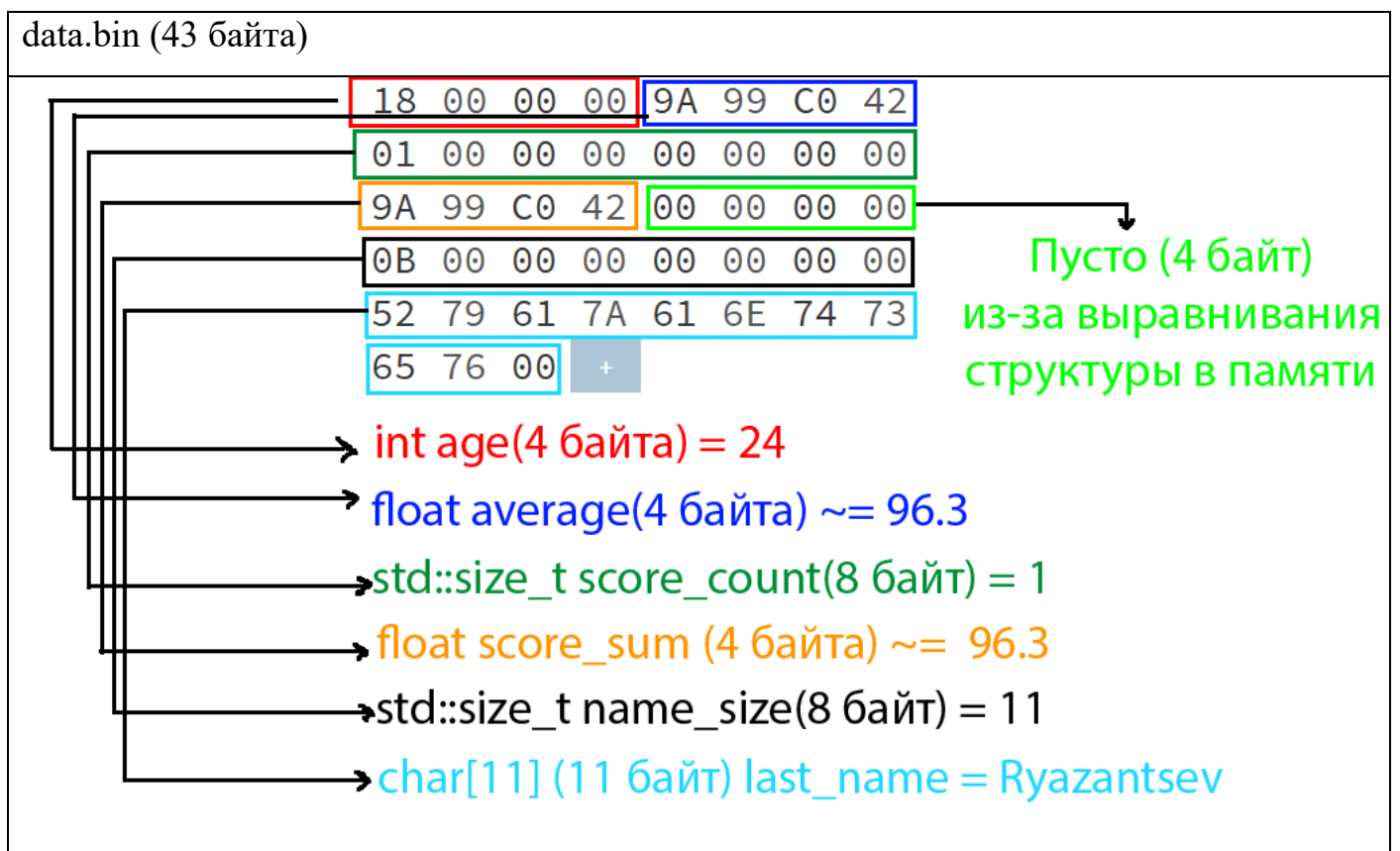
Для класса из лабораторной работы №2 перегрузить операции ввода/вывода, позволяющие осуществлять ввод и вывод в удобной форме объектов классов:

- вывод объекта класса в текстовый файл;
- вывод объекта класса в двоичный файл;
- ввод объекта класса из двоичного файла.

Дополнить демонстрационную программу, продемонстрировав все перегруженные операции.

С помощью любого шестнадцатеричного редактора (например, можно использовать online-редактор <http://hexed.it>) открыть и посмотреть текстовый и двоичный файлы, созданные программно.

Сравнить содержимое файлов, обратить внимание, как представлены данные одного и того же типа в обоих файлах.



data.txt (109 байт)

7B	22	5F	6C	61	73	74	5F	{ "_last_
6E	61	6D	65	22	3A	20	22	name": "
52	79	61	7A	61	6E	74	73	Ryazants
65	76	22	2C	20	22	5F	61	ev", "_a
67	65	22	3A	22	32	34	22	ge": "24"
2C	20	22	5F	61	76	65	72	, "_aver
61	67	65	5F	73	63	6F	72	age_scor
65	22	3A	22	39	36	2E	33	e": "96.3
30	22	2C	20	22	5F	73	63	0", "_sc
6F	72	65	5F	73	75	6D	22	ore_sum"
3A	22	39	36	2E	33	30	22	: "96.30"
2C	20	22	5F	73	63	6F	72	, "_scor
65	5F	63	6F	75	6E	74	22	e_count"
3A	22	31	22	7D	+			: "1"}

```
{"_last_name": "Ryazantsev", "_age": "24", "_average_score": "96.30",  
"_score_sum": "96.30", "_score_count": "1"}
```

Исходный код

"student_data.h"

```
#ifndef S3L1_STUDENT_DATA_H
```

```
#define S3L1_STUDENT_DATA_H
```

```
#include <cstdint>
```

```
#include <ostream>
```

```
#define S3L1_STUDENT_DATA_DEFAULT_PRINT_NAME "NULL"
```

```
namespace s3l1 {
```

```
    class StudentData {
```

```
    public:
```

```
        StudentData();
```

```
        StudentData(const char* last_name, int age = 0, float average_score = 0.0);
```

```
        ~StudentData();
```

```
        StudentData(const StudentData &sd);
```

```
        StudentData(StudentData &&sd);
```

```
        StudentData& operator=(const StudentData& sd);
```

```
        StudentData& operator=(StudentData &&sd);
```

```
        StudentData& operator++();
```

```
        StudentData operator++(int);
```

```
        StudentData operator+(float score);
```

```
        operator const char*();
```

```
        friend StudentData operator-(StudentData& data, float score);
```

```
        friend std::ofstream& operator<<(std::ofstream& out, StudentData& data);
```

```
        friend std::ifstream& operator>>(std::ifstream& in, StudentData& data);
```

```
        const char* get_last_name() const;
```

```
        int get_age() const;
```

```
        float get_average_score() const;
```

```
        bool set_last_name(const char* name);
```

```
        bool set_age(int age);
```

```
        bool set_average_score(float score);
```

```
        void print_last_name() const;
```

```
        void print_age() const;
```

```
        void print_average_score() const;
```

```
        void print_data() const;
```

```
        StudentData& binary_mode(bool is_binary);
```

```
    private:
```

```
        char* _last_name;
```

```
        int _age;
```

```
        float _average_score;
```

```
        float _score_sum;
```

```
        std::size_t _score_count;
```

```
    /*
```

по сути, это временное хранилище нашего json из приведения типов.
сделано оно для того, чтобы я мог принтить его достаточно спокойно
но придётся держать в уме, что поинтеры на неё удалятся после

деструктора,

и если кто вздумает хранить приведённый тип, то он расстроится

```

    */
    char* _as_json;

    bool _ofstream_binary;

    bool _is_valid_name(const char* name) const;
    bool _is_valid_age(const int age) const;
    bool _is_valid_average(const float average) const;

    void _copy_string(char** dest,const char* src);
    void _update_json();
    static std::size_t _get_true_size(const char* str);

};

}

#endif //S3L1_STUDENT_DATA_H

```

"student_data.cpp"

```
#include "student_data.h"
```

```
#include <cstdint>
```

```
#include <cmath>
```

```
#include <cstdio>
```

```
#include <cstring>
```

```
#include <iostream>
```

```
#include <fstream>
```

```
namespace s3l1 {
```

```
    const char* StudentData::get_last_name() const {  
        return _last_name;  
    }
```

```
    int StudentData::get_age() const {  
        return _age;  
    }
```

```
    float StudentData::get_average_score() const {  
        return _average_score;  
    }
```

```
    bool StudentData::set_last_name(const char* name) {  
        if (!_is_valid_name(name)) {  
            return false;  
        }
```

```
        _copy_string(&_amp;_last_name,name);
```

```
        return true;  
    }
```

```
    bool StudentData::set_age(int age) {  
        if (!_is_valid_age(age)) return false;  
        _age = age;  
        return true;  
    }
```

```
    bool StudentData::set_average_score(float score) {  
        if (!_is_valid_average(score)) return false;  
        _score_count = 1;  
        _score_sum = score;  
        _average_score = score;  
        return true;  
    }
```

```
    StudentData::StudentData():  
        _last_name(NULL),  
        _age(0),  
        _average_score(0.0),  
        _score_count(0),  
        _score_sum(0.0),  
        _as_json(NULL),  
        _ofstream_binary(false)  
    {}
```

```
    StudentData::StudentData(const char* last_name, int age, float average_score):  
    StudentData() {  
        set_last_name(last_name);  
        set_age(age);
```

```

        set_average_score(average_score);
    }

StudentData::~~StudentData() {
    if (_last_name) {
        delete[] _last_name;
        _last_name = NULL;
    }
    if (_as_json) {
        delete[] _as_json;
        _as_json = NULL;
    }
}

StudentData::StudentData(const StudentData &sd): StudentData() {
    _age = sd._age;
    _average_score = sd._average_score;
    _copy_string(&_amp;_last_name, sd._last_name);

    _score_count = sd._score_count;
    _score_sum = sd._score_sum;
    _as_json = NULL;
}

StudentData::StudentData(StudentData &&sd): StudentData() {
    _age = sd._age;
    _average_score = sd._average_score;
    _last_name = sd._last_name;
    sd._last_name = NULL;

    _score_count = sd._score_count;
    _score_sum = sd._score_sum;
}

StudentData& StudentData::operator=(const StudentData& sd) {
    if (&sd == this) return *this;

    _age = sd._age;
    _average_score = sd._average_score;
    _copy_string(&_amp;_last_name, sd._last_name);
    _score_count = sd._score_count;
    _score_sum = sd._score_sum;

    if (_as_json) {
        delete [] _as_json;
        _as_json = NULL;
    }

    return *this;
}

StudentData& StudentData::operator=(StudentData &&sd) {
    if (&sd == this) return *this;

    _age = sd._age;
    _average_score = sd._average_score;
    if (_last_name) {
        delete [] _last_name;
    }
    _last_name = sd._last_name;
    sd._last_name = NULL;
}

```

```

        _score_count = sd._score_count;
        _score_sum = sd._score_sum;

        return *this;
    }

    void StudentData::print_last_name() const {
        const char* to_print = _last_name ? _last_name :
S3L1_STUDENT_DATA_DEFAULT_PRINT_NAME;
        std::cout << to_print << std::endl;
    }

    void StudentData::print_age() const {
        std::cout << _age << std::endl;
    }

    void StudentData::print_average_score() const {
        char buff[16]; sprintf(buff, "%.2f", _average_score);
        std::cout << buff << std::endl;
    }

    void StudentData::print_data() const {
        std::cout << "Student: ";
        print_last_name();
        std::cout << "\tAge: ";
        print_age();
        std::cout << "\tAverage Score: ";
        print_average_score();
    }

    bool StudentData::_is_valid_name(const char* name) const {
        if (!name) {
            return false;
        }
        return (bool)(*name != 0);
    }

    bool StudentData::_is_valid_age(const int age) const {
        /*
        понятно, что студент вряд ли будет младенцем или старичком 100+,
        но кто-то, например, может быть зачислен с рождения (как в гарри поттере,
да)

        или числиться в базах до смерти,
        и мы не хотим вывалиться в ошибку по недосмотру
        */
        return age >= 0 && age <= 150;
    }

    /*
    система оценки не указана, так что просто проверим что это вообще число,
    что оно больше нуля,
    и далее отдадим всё в руки того, кто будет дёргать за ручки нашего интерфейса.
    */
    bool StudentData::_is_valid_average(const float average) const {
        if (std::isnan(average) || std::isinf(average)) {
            return false;
        }
        return average >= 0.0;
    }

    void StudentData::_copy_string(char** dest, const char* src) {

```



```

    if (!dest) return;
    if (*dest == src) return;

    if (*dest) {
        delete[] *dest;
        *dest = NULL;
    }

    if (src) {
        std::size_t size = _get_true_size(src);
        *dest = new char[size];
        memcpy(*dest, src, size);
        (*dest)[size-1] = 0;
    }
}

StudentData& StudentData::operator++() {
    _age++;
    return *this;
}

StudentData StudentData::operator++(int) {
    StudentData temp = *this;
    _age++;
    return temp;
}

StudentData StudentData::operator+(float score) {
    StudentData new_sd(*this);

    new_sd._score_count+=1;
    new_sd._score_sum+=score;
    new_sd._average_score = new_sd._score_sum/(float)new_sd._score_count;

    return new_sd;
}

StudentData operator-(StudentData& lv, float score) {
    /*
    благодаря этой штучке мы не улетим в отрицательные значения
    если на фронте кто-то будет усердно вычитать
    */
    StudentData new_sd(lv);
    float subtract = score <= new_sd._average_score ? score :
new_sd._average_score;
    new_sd._average_score -= subtract;
    new_sd._score_sum = new_sd._average_score*new_sd._score_count;
    return new_sd;
}

StudentData::operator const char *() {
    _update_json();
    return _as_json;
}

void StudentData::_update_json() {
    char new_json[BUFSIZ] = {0};
    /*
    Делаем все прошлые pointers инвалидными, чтобы жизнь мёдом не казалась тем
кто копировал бездумно.
    */
    if (_as_json) {

```

```

        delete[] _as_json;
        _as_json = NULL;
    }

    memset(new_json, 0, BUFSIZ);

    std::size_t new_size = snprintf(new_json, BUFSIZ-1,
        "{ \"_last_name\": \"%s\", \"_age\": \"%d\", \"_average_score\": \"%.2f\", \"_score_sum\": \"%.2f\", \"_score_count\": \"%ld\"}",
        _last_name ? _last_name : S3L1_STUDENT_DATA_DEFAULT_PRINT_NAME,
        _age,
        _average_score,
        _score_sum,
        _score_count
    );

    _as_json = new char[new_size+1];
    memcpy(_as_json, new_json, new_size+1);
}

std::size_t StudentData::_get_true_size(const char* str) {
    if (!str) return 0;

    const char* start = str;
    while (*(str++));
    return str-start;
}

StudentData& StudentData::binary_mode(bool is_binary) {
    _ofstream_binary = is_binary;
    return *this;
}

std::ofstream& operator<<(std::ofstream& out, StudentData& data) {
    std::cout << data._ofstream_binary;
    if (data._ofstream_binary) {
        std::size_t name_size = StudentData::_get_true_size(data._last_name);
        struct save_template {
            int age;
            float average;
            std::size_t score_count;
            float score_sum;
            std::size_t name_size;
        } static_data {
            data._age,
            data._average_score,
            data._score_count,
            data._score_sum,
            name_size,
        };
        out.write((char*)&static_data, sizeof(static_data));
        if (name_size > 1) {
            out.write(data._last_name, name_size);
        }
    } else {
        //Если тут не привести тип, мы в бесконечную рекурсию упадём.
        out << (const char*) data;
    }

    return out;
}

```

```

std::ifstream& operator>>(std::ifstream& in, StudentData& data) {
    struct save_template {
        int age;
        float average;
        std::size_t score_count;
        float score_sum;
        std::size_t name_size;
    } static_data = {};
    char* buffer = NULL;

    in.read((char*)&static_data, sizeof(static_data));
    if (static_data.name_size > 1) {
        buffer = new char[static_data.name_size];
        in.read(buffer, static_data.name_size);
        buffer[static_data.name_size-1] = 0;
    }

    data = StudentData(buffer, static_data.age, static_data.average);
    data._score_count = static_data.score_count;
    data._score_sum = static_data.score_sum;
    if (buffer) {
        delete [] buffer;
    }
    return in;
}
}

```

"main.cpp"

```
#include "student_data.h"
#include <fstream>
#include <iostream>

void print_title(const char* title) {
    std::cout << "\033[32m|\\" << title << " |\\|\033[0m" << std::endl;
}

void print_closing(const char* title) {
    std::cout << "\033[32m|^| " << title << " |^|\033[0m" << std::endl;
}

void print_valid_example(s311::StudentData& data) {
    print_title("Setters example (valid)");
    std::cout << "\tSET: Last name: Zaharov" << std::endl;
    std::cout << "\tSET: Age: 33" << std::endl;
    std::cout << "\tSET: Average score: 68.8" << std::endl << std::endl;

    data.set_last_name("Zaharov");
    data.set_age(33);
    data.set_average_score(68.8);
    data.print_data();

    print_closing("Setters example (valid) end");
}

void print_invalid_example(s311::StudentData& data) {
    print_title("Setters example (invalid)");
    std::cout << "\tSET: Last name: \"\"" << std::endl;
    std::cout << "\tSET: Age: -6" << std::endl;
    std::cout << "\tSET: Average score: -88.5" << std::endl << std::endl;

    data.set_last_name("");
    data.set_age(-6);
    data.set_average_score(-88.5);
    data.print_data();
    print_closing("Setters example (invalid) end");
}

s311::StudentData setup_copy() {
    s311::StudentData source("Ryazantsev", 24, 96.3);
    s311::StudentData dest(source);
    return dest;
}

void print_simple_part() {
    s311::StudentData student_simple{};
    print_title("Simple constructor");
    student_simple.print_data();
    print_closing("Simple constructor end");
    print_valid_example(student_simple);
    print_invalid_example(student_simple);
}

void print_various_constructors_part() {
    print_title("Full constructor");
    s311::StudentData student_all_fields("Ryazantsev", 24, 96.3);
    student_all_fields.print_data();
    print_closing("Full constructor end");
}
```

```

    print_title("Full constructor (invalid)");
    s3l1::StudentData student_incorrect("", -444, -3.2);
    student_incorrect.print_data();
    print_closing("Full constructor (invalid) end");

    print_title("Copy constructor (Ryazantsev to new)");
    std::cout << "Original initialized out of scope and is deleted by this point"
<< std::endl;
    /*
    Чтобы показать копирование значения, а не pointers,
    вызываю отдельную функцию, по истечении которой исходник удалится.
    */
    s3l1::StudentData student_copied = setup_copy();
    student_copied.print_data();
    print_closing("Copy constructor end");

    print_title("Move constructor (From Ryazantsev)");
    s3l1::StudentData student_moved = std::move(student_all_fields);
    std::cout << "Destination: " << std::endl;
    student_moved.print_data();
    std::cout << "Source: (will be deleted after function's return)" << std::endl;
    student_all_fields.print_data();
    print_closing("Move constructor end");
}

void print_getters() {
    print_title("Getters example");
    s3l1::StudentData student("Ryazantsev", 24, 96.3);
    student.print_data();
    std::cout << std::endl;

    std::cout << "student.get_last_name(): " << student.get_last_name() <<
std::endl;
    std::cout << "student.get_age(): " << student.get_age() << std::endl;
    std::cout << "student.get_average_score(): " << student.get_average_score() <<
std::endl;
    print_closing("Getters example end");
}

void print_prints() {
    print_title("Print functions example");

    s3l1::StudentData student("Ryazantsev", 24, 96.3);

    std::cout << "student.print_data():" << std::endl;
    student.print_data();
    std::cout << "student.print_last_name():" << std::endl;
    student.print_last_name();
    std::cout << "student.print_age():" << std::endl;
    student.print_age();
    std::cout << "student.print_average_score():" << std::endl;
    student.print_average_score();

    print_closing("Print functions end");
}

void print_move_assignment() {
    print_title("Move assignment example");

    s3l1::StudentData s1("Ryazantsev", 24, 96.3);
    s3l1::StudentData s2("Zaharov", 33, 68.8);
    char buff[16];

```

```

std::cout << "Start values" << std::endl;
std::cout << "s1:" << std::endl;
s1.print_data();
std::cout << "s2:" << std::endl;
s2.print_data();

sprintf(buff, "%p", s1.get_last_name());
std::cout << std::endl << "s1 name pointer: " << buff << std::endl;
sprintf(buff, "%p", s2.get_last_name());
std::cout << "s2 name pointer: " << buff << std::endl;

std::cout << "s1 = std::move(s2)" << std::endl;
s1 = std::move(s2);
std::cout << "s1:" << std::endl;
s1.print_data();
std::cout << "s2:" << std::endl;
s2.print_data();

sprintf(buff, "%p", s1.get_last_name());
std::cout << std::endl << "s1 name pointer: " << buff << std::endl;
sprintf(buff, "%p", s2.get_last_name());
std::cout << "s2 name pointer: " << buff << std::endl;

print_closing("Move assignment example end");
}

void print_copy_assignment() {
    print_title("Copy assignment example");
    s3l1::StudentData s1("Ryazantsev", 24, 96.3);
    s3l1::StudentData s2("Zaharov", 33, 68.8);

    std::cout << "Start values" << std::endl;
    std::cout << "s1:" << std::endl;
    s1.print_data();
    std::cout << "s2:" << std::endl;
    s2.print_data();

    std::cout << "s1 = s2" << std::endl;
    s1 = s2;

    std::cout << "s1:" << std::endl;
    s1.print_data();
    std::cout << "s2:" << std::endl;
    s2.print_data();

    char buff[16];
    sprintf(buff, "%p", s1.get_last_name());
    std::cout << std::endl << "s1 name pointer: " << buff << std::endl;
    sprintf(buff, "%p", s2.get_last_name());
    std::cout << "s2 name pointer: " << buff << std::endl;
    print_closing("Copy assignment example end");
}

void print_assignment() {
    print_move_assignment();
    print_copy_assignment();
}

void print_increment() {
    print_title("Increments overloading example");

```

```

s3l1::StudentData student("Ryazantsev", 24, 96.3);
std::cout << "Start values: " << std::endl;
student.print_data();
std::cout << "\n(++student).print_data()" << std::endl;
(++student).print_data();
std::cout << "\n(student++).print_data()" << std::endl;
(student++).print_data();
std::cout << "\nEnd values:" << std::endl;
student.print_data();
print_closing("Increments overloading end");
}

void print_addition() {
    print_title("Addition overloading example");
    s3l1::StudentData student("Ryazantsev", 24, 96.3);
    std::cout << "Start values:" << std::endl;
    student.print_data();
    std::cout << "\nstudent = student+0.0" << std::endl;
    std::cout << "Expected: average score = (96.3+0.0)/2 = 48.15" << std::endl;
    student = student+0.0;
    student.print_data();
    std::cout << "\nstudent = student+55.4" << std::endl;
    std::cout << "Expected: average score = (96.3+55.4)/3 ~= 50.57" << std::endl;
    student = student+55.4;
    student.print_data();
    print_closing("Addition example end");
}

void print_subtraction() {
    print_title("Subtraction overloading example");
    s3l1::StudentData student("Ryazantsev", 24, 96.3);
    student = (student + 0.0) + 55.4;
    std::cout << "Start values:" << std::endl;
    student.print_data();

    std::cout << "\nstudent = student-0.0" << std::endl;
    std::cout << "Expected: average score = 50.57" << std::endl;
    student = student - 0.0;
    student.print_data();

    std::cout << "\nstudent = student - 15.52" << std::endl;
    std::cout << "Expected: average score = 35.05" << std::endl;
    student = student - 15.52;
    student.print_data();

    std::cout << "\nstudent = student - 88.3" << std::endl;
    std::cout << "Expected: average score = 0.0" << std::endl;
    student = student - 88.3;
    student.print_data();

    print_closing("Subtraction overloading end");
}

void print_conversion() {
    print_title("Implicit conversion overloading example");
    s3l1::StudentData student("Ryazantsev", 24, 96.3);
    std::cout << "Start values" << std::endl;
    student.print_data();

    std::cout << "std::cout << student << std::endl;" << std::endl;
    std::cout << "Expected: Implicit conversion of student object into JSON string"
<< std::endl;

```

```

        std::cout << student << std::endl;

        print_closing("Implicit conversion end");
    }

void print_files() {
    print_title("Outputting to files: data.txt, data.bin");
    s311::StudentData student("Ryazantsev", 24, 96.3);
    std::ofstream text("data.txt", std::ios::out);
    if (text) {
        text << student;
        text.close();
    } else {
        std::cerr << "Cannot open file \"data.txt\" in write mode" << std::endl;
    }

    std::ofstream bin("data.bin", std::ofstream::out | std::ofstream::binary);
    if (bin) {
        bin << student.binary_mode(true);
        bin.close();
    } else {
        std::cerr << "Cannot open file \"data.bin\" in write mode" << std::endl;
    }
    print_closing("Outputting to files end");
}

void load_from_binary() {
    print_title("Loading from binary: data.bin");
    s311::StudentData new_student{};
    std::cout << "Before load: " << std::endl;
    new_student.print_data();
    std::ifstream bin("data.bin", std::ifstream::in | std::ifstream::binary);
    if (bin) {
        /*
        он может выбросить тут эксепшн аж из ofstream.read(),
        но эта ситуация произойдёт только если в самом ofstream
        __forced_unwind == true, чего у нас нет, а если бы было -
        ну, судя по названию, правильно было бы чтоб всё сломалось.
        */
        bin >> new_student;
        std::cout << "After load: " << std::endl;
        new_student.print_data();
    } else {
        std::cerr << "Cannot open file \"data.bin\" in read mode. Does it exist?"
<< std::endl;
    }

    print_closing("Loading from binary end");
}

int main(void) {
    print_simple_part();
    print_various_constructors_part();
    print_getters();
    print_prints();
    print_assignment();
    print_increment();
    print_addition();
    print_subtraction();
    print_conversion();
    print_files();
    load_from_binary();
}

```



```
    return 0;  
}
```

Выводы

В процессе выполнения задания добавлены перегрузки операторов '<<' и '>>' как дружественные функции классов выходного и входного файловых потоков соответственно. Поскольку класс потока не хранит данные о режиме открытия, а применяет необходимые модификаторы непосредственно в момент открытия файла, возникла проблема с отсутствием возможности понять в момент записи предполагаемый выходной формат (двоичный или же строковый). Вследствие этого, в класс было добавлено поле `_ofstream_binary` и функция `binary_mode()`, возвращающая ссылку на объект класса, из которого и была вызвана. Благодаря данным изменениям, был, в соответствии с заданием, реализован "ввод-вывод в удобной форме", позволяющий перед началом вывода единоразово вызвать `binary_mode()`, и далее свободно сериализовать класс в двоичный или строковый формат.

В процессе тестирования также было замечено, что сохранение в виде приведённой к однобайтовому указателю структуры оставляет часть байт неинициализированными. Это происходит вследствие привязки размера структуры компилятором к определённой "кратности", и несёт за собой возможные последствия в виде некорректной инициализации и потери данных при попытке перевода файлов с данными текущей версии на произвольный новый формат сохранения. Поскольку стандарт C++ не предполагает функционала компрессии структур, для обеспечения совместимости с любым компилятором было принято решение предварительно инициализировать структуру, заполняя все байты 0.

Отметим, что поведение оператора '<<' входных и выходных потоков без переопределения предполагает приведение правого значения (нашего класса) именно к строке, но явное переопределение для классов файловых потоков имеет больший приоритет и исполняется вместо приведения, которое также имеет перегрузку в реализованном классе.